



Blockchain

Blockchain to sekwencja rekordów, zwanych blokami, powiązanych w łańcuch. Klucowym elementem w takim łańcuchu pełni przyjęta a-priori funkcja hashująca H .

Funkcja hashująca potrafi zamienić dowolny ciąg znaków na pojedynczą liczbę (zwaną hashem), w taki sposób aby funkcję odwrotną było bardzo trudno obliczyć. Innymi słowy, znając sam hash $H(x)$ trudno jest znaleźć ciąg znaków x , który go generuje. Z drugiej strony, nawet niewielka zmiana oryginalnego ciągu powodować powinna dużą i nieprzewidywalną zmianę w hashu.

Dzięki temu, jeśli znamy oczekiwany hash danej wiadomości, możemy łatwo potwierdzić czy dana wiadomość nie uległa zmianie.

Każdy blok w łańcuchu blockchain zawiera:

- Hash swojego poprzednika. Hash ten pełni rolę wskaźnika, jednoznacznie określając który blok jest tym poprzednim. Ponadto, pełni on też rolę sprawdzającą – zawartość poprzedniego bloku nie może zostać zmieniona, gdyż wtedy zmianie uległby hash i lista zostałaby przerwana.
- Dowolną wartość seed, spełniającą warunek trudności (o czym za chwilę)
- Dowolną pozostałą treść, zależną od tego co chcemy przechowywać w rekordach.

Często w bloku przechowywane są dodatkowe informacje, jak np. data powstania danego bloku, bądź identyfikator osoby która znalazła wartość seed.

Dla danego blockchainu określa się trudność T wyrażoną dodatnią liczbą całkowitą. Dana wartość T to liczba początkowych bitów hashu która musi być równa 0 dla każdego bloku.

Użytkownik chcący dodać nowy element do blockchainu musi wykonać trudne zadanie obliczeniowe, jakim jest dobranie odpowiedniej liczby seed, tak aby wartość H dla całego bloku spełniała trudność T . W ten sposób wszyscy użytkownicy blockchainu muszą ciężko pracować by dodawać kolejne elementy, a kolejne wpisy pojawiają się w sposób nieprzewidywalny. Nie ma jednej uprzywilejowanej osoby która mogłaby dowolnie dodawać elementy to ciągu.

Karty graficzne, ze względu na swoją dużą moc obliczeniową, są dobrymi kandydatami na maszyny wyszukujące wartości seed celem dodania nowych elementów do blockchainu.

Twoim zadaniem jest napisanie programu do budowania takiego łańcucha blockchain. Na potrzeby zadania używać będziemy dość starej już funkcji hashującej `md5`, która koduje wiadomość w liczbie 128-bitowej. W dzisiejszych standardach ta funkcja jest zbyt słaba – zbyt łatwo ją odwrócić i znaleźć nową wiadomość o tej samej wartości hash. Niemniej powszechnie stosowane funkcje, jak np. `sha256` działają w bardzo podobny sposób, są one tylko trochę bardziej skomplikowane.

Twoje rozwiązanie zostanie skompilowane kompilatorem `nvcc` z flagami `-arch sm_61 -O3 --extended-lambda` i uruchomione na karcie GTX 1060.

Wejście

W pierwszej linii wejścia podane są dwie liczby całkowite z oraz t .

- z ($2 \leq z \leq 1000$) oznacza liczbę bloków do wygenerowania. Każdy kolejny blok powinien wskazywać na poprzedni.

- t ($1 \leq t \leq 63$) oznacza trudność – liczbę początkowych bitów, które muszą być równe 0 dla każdego hasha w ciągu.

W każdej z kolejnych z linii znajduje się ciąg znaków, o maksymalnej długości 100 000. Są to kolejne wiadomości (bez znaku nowej linii), które mają być umieszczone w blokach.

Na blok składać się będzie:

- pierwsze 128 bitów – hash md5 poprzedniego bloku. Dla pierwszego bloku jest to liczba 0;
- kolejne 128 bitów – dowolna liczba seed zapewniająca, że hash md5 całego bloku spełnia warunek trudności;
- kolejne $8n$ bitów – wiadomość wczytana ze standardowego wejścia (gdzie n to liczba znaków w wiadomości⁴).

Wyjście

Twój program powinien wypisać z linii. W i -tej spośród nich powinien znaleźć się seed dla i -tego bloku w wyliczonym przez Ciebie ciągu blockchain. Seed należy wypisać jako liczbę hexadecymalną, łącznie z wiodącymi zerami, bez żadnych prefixów (np. `0x`).

Przykład

Dla danych wejściowych	jedną z możliwych wyników jest
<pre> 5 19 Hello world! --> foo <-- ~!@#\$%^&*()_+`1234567890-=QWERTYUIOP[]qwertyuiop78 (Druga wiadomość zawiera po dwie spacje na początku i na końcu. Czwarta wiadomość to ciąg pusty) </pre>	<pre> 0000000056420200000000000000000000 00000000e6101200000000000000000000 00000000c9a90100000000000000000000 0000000007840300000000000000000000 00000000ce3f1000000000000000000000 </pre>

Dla powyższego układu blockchain wygląda następująco:

Blok 1 to dwie liczby 128-bitowe oraz wiadomość, w całości reprezentowany przez następujący ciąg hexadecymalny:

[illegible]

Który dla czytelności przedstawimy jako:

```
[00000000000000000000000000000000] [00000000564202000000000000000000] [Hello
world!]
```

i wartość md5 tego ciągu bitów wynosi

00000d6888740b7292668ac332b5dc15



Kolejne bloki to:

```
blok 2 = [00000d6888740b7292668ac332b5dc15] [00000000e61012000000000000000000] [  
--> foo <-- ]  
md5 bloku 2 = 000012f5cda3d2a96353b08540564360  
blok 3 = [000012f5cda3d2a96353b08540564360] [00000000c9a901000000000000000000] [~!@#$$%^&*()_+`  
md5 bloku 3 = 000011afc79402d2777b256bc05aea39  
blok 4 = [000011afc79402d2777b256bc05aea39] [00000000788403000000000000000000] []  
md5 bloku 4 = 000014a8e2404111a732c67c03cde144  
blok 5 = [000014a8e2404111a732c67c03cde144] [00000000ce3f100000000000000000000] [| |]  
md5 bloku 5 = 00000144aa64aa16673b36c74248860a
```